

Proyecto Aplicado 2

Modelación con integración para resolver problemas de decisión

Modalidad Trabajo individual con base de datos entregada

Periodo Semanas 10 a 15

Valor 11 % de la nota final

Producto Ficha técnica, código en Python, aplicación web, resumen ejecutivo y defensa oral

Docente Francisco Campos Sandi

Proyecto asignado

Estudiante	VILLALOBOS VILLALTA KENDALL FABRICIO
Código	C-09
Proyecto	2. Cafeína, atención y sueño en semana de exámenes (página 6)
Archivo de datos	P02_C-09_VILLALOBOS.xlsx

Lineamientos generales

Naturaleza del proyecto

Cada estudiante asumirá el papel de una persona analista de datos que trabaja para una organización situada en un lugar real de Costa Rica para resolver un problema concreto de decisión. **No es necesario conseguir datos:** el docente entrega a cada estudiante un archivo de Excel propio con la base de datos del proyecto asignado (entre 1000 y 1500 registros), lista para trabajar, con una hoja **Diccionario** que explica el significado y las unidades de cada columna. El catálogo reúne diez problemas poco comunes sobre el **cerebro y el comportamiento** (proyectos 1, 2 y 5), los **sismos y volcanes** (3 y 4), las **aves migratorias** (6), el **mar del Golfo Dulce** (7, 8 y 10) y la **energía del viento** (9). En todos, la herramienta central es la **integral**: áreas, acumulación, valor promedio, integrales impropias e integración numérica. El trabajo deberá producir una **solución utilizable**: una recomendación respaldada por los datos, un modelo matemático basado en integración, un programa en Python verificable y una aplicación web funcional.

Sobre los datos

Los lugares e instituciones mencionados son reales, pero **los datos son simulados con fines didácticos**, con valores realistas para cada lugar. No son registros oficiales de esas instituciones y no deben citarse como tales en el informe ni en la presentación; para el marco teórico y el contexto deben usarse fuentes bibliográficas reales.

Trabajo estrictamente individual

La exploración de los datos, la modelación, la programación, el diseño visual, el análisis, la presentación y la defensa serán realizados por una sola persona. Cada estudiante recibe una base **distinta**, por lo que sus resultados numéricos son únicos: no se permitirán productos compartidos, código idéntico ni visualizaciones duplicadas.

Requisitos del análisis

1. Explorar la base entregada: periodo, unidades, patrones (por hora, día, semana o temporada) y relaciones entre variables.
2. Detectar y tratar registros faltantes o atípicos, como ocurre con datos reales, documentando el criterio usado.
3. Formular una pregunta que implique **decidir**, no solamente describir.
4. Construir un modelo que relacione tasas, acumulación, áreas, cambio total o valor presente.
5. Verificar el modelo (por ejemplo, comparando la integral con el acumulado observado) y comparar al menos tres escenarios factibles.
6. Entregar una recomendación cuantificada, aplicable y acompañada de sus condiciones y limitaciones.

Prácticas no admitidas

No se admitirá eliminar, modificar o completar datos sin justificarlo; usar la base de otra persona; ejercicios de libro disfrazados de proyecto; integrales sin interpretación; código que no se ejecute, que no lea la base entregada o que el estudiante no pueda explicar; ni recomendaciones que no se desprendan del análisis.

Del dato a la integral: ruta metodológica común

Todos los proyectos siguen la misma ruta. Los datos entregados son **discretos** (por intervalo, día o lote), por lo que deberá justificarse cómo se pasa de ellos a una función o a una aproximación de la integral.



Cuando la tasa se ajuste a una función, deberá integrarse analíticamente (sustitución, partes o fracciones parciales, según corresponda). Cuando se trabaje con datos tabulados, se usará la regla del trapecio o de Simpson y se comparará con el resultado del modelo ajustado. En ambos casos se reportarán las **unidades** de la integral y su interpretación.

Parte programada en Python (obligatoria)

Todo el procesamiento de la base, la modelación, las integrales, los escenarios y la aplicación web se programarán en **Python 3.10 o superior**. Cada proyecto incluye un recuadro *Parte programada en Python* con las funciones mínimas que deben implementarse y la validación obligatoria; además, todo proyecto debe cumplir lo siguiente.

Requisitos comunes del código

1. **Lectura directa.** El Excel entregado se lee con `pandas.read_excel(ruta, sheet_name=None)`; no se copian valores a mano ni se modifica el archivo original.
2. **Funciones documentadas.** Cada función tiene un *docstring* con entradas, salidas y **unidades**; los parámetros (tarifas, costos, umbrales) se leen de la base, no se escriben dentro del código.
3. **Bibliotecas.** `numpy` y `pandas`; `matplotlib` o `plotly` para los gráficos; `scipy.integrate` (`trapezoid`, `simpson`, `quad`) solo para comparar.
4. **Métodos propios.** Las reglas del trapecio y de Simpson, la integral acumulada y, si el proyecto lo pide, la integración paso a paso o la bisección se programan desde cero; toda integral importante se obtiene de dos formas (dos reglas, o regla y fórmula exacta) y se reporta el error relativo.
5. **Calidad del código.** Anotaciones de tipo, estilo PEP 8, funciones cortas con una sola responsabilidad, operaciones vectorizadas con `numpy/pandas` donde sea posible y, al menos, una clase propia.
6. **Pruebas.** Al menos ocho pruebas con `pytest` que verifiquen integrales de funciones con resultado conocido, unidades, balances (por ejemplo, acumulado igual a la suma) y casos límite.
7. **Control de versiones.** *Fork* del repositorio del curso con al menos 10 *commits* distribuidos en las semanas del proyecto; el historial forma parte de la evidencia de autoría.
8. **Gráficos.** Con título, ejes rotulados y unidades; las áreas que representan integrales se somborean.
9. **Reproducibilidad.** `requirements.txt`, `README.md` con instrucciones y un cuaderno `analisis.ipynb` que se ejecute de principio a fin sin errores.

Estándar profesional del proyecto

1. **Integración continua.** Un flujo de GitHub Actions (`.github/workflows/pruebas.yml`) ejecuta `pytest` en cada *push*; el `README.md` muestra la insignia del resultado.
2. **Estilo verificable.** El código pasa `ruff check` (o `flake8`) sin advertencias y se formatea con `ruff format` o `black`.
3. **Validación de entradas.** Al leer el Excel se verifican hojas, columnas y unidades, con mensajes de error claros; no hay rutas absolutas ni valores escritos a mano en el código.
4. **Resultados reproducibles.** Un comando (`python -m src.reporte`) regenera todas las

tablas y figuras de la ficha y de las diapositivas en la carpeta **resultados/**.

5. **Documentación.** Diccionario de datos y bitácora de supuestos en **docs/**; cada figura de la presentación indica su fuente y unidades.
6. **Comunicación ejecutiva.** Diapositivas con una idea por lámina, títulos que enuncian la conclusión y la recomendación en la primera y en la última lámina.

Estructura mínima del repositorio

datos/	Archivo de Excel entregado, sin modificar.
src/limpieza.py	Detección y tratamiento de faltantes y atípicos.
src/modelo.py	Ajuste de funciones, integrales y verificación.
src/escenarios.py	Alternativas, sensibilidad y recomendación.
src/reporte.py	Genera tablas y figuras en resultados/ .
tests/	Pruebas automáticas con pytest .
analisis.ipynb	Análisis completo con gráficos y explicación.
app.py	Aplicación web que importa las funciones de src/ .
docs/	Diccionario de datos y bitácora de supuestos.
informe/	Ficha técnica y resumen ejecutivo (PDF).
presentacion/	Diapositivas en PDF, con el marco teórico.
.github/workflows/	Integración continua que ejecuta las pruebas.
requirements.txt, README.md	Dependencias, instrucciones y enlace a la aplicación.

Aplicación web en Python

La aplicación (**app.py**) se construirá con **Streamlit** (recomendado), Dash, Panel o Shiny para Python y se publicará con un enlace público (Streamlit Community Cloud, Hugging Face Spaces u otro servicio). Deberá leer la misma base de Excel y permitir **cargar otra base con la misma estructura**, usar caché para los cálculos pesados, permitir descargar los resultados en CSV, modificar al menos **tres parámetros** con controles (deslizadores, listas o casillas), mostrar el escenario base y dos alternativos, actualizar gráficos e indicadores al instante, sombrear las áreas que representan las integrales y destacar la recomendación, de modo que la persona usuaria comprenda el problema sin leer el código.

Entregables

1. **Código en Python** completo y propio, con pruebas, según la estructura mínima, subido al repositorio del curso (ver recuadro).
2. **Presentación** de 12 a 15 minutos con 10 a 14 diapositivas (PDF), que incluya un **marco teórico**: los conceptos de cálculo integral y del área de aplicación que se necesitan para la solución, con las fórmulas que se usan, su deducción breve o el teorema que las respalda, y las referencias bibliográficas.
3. Ficha técnica de modelación de 4 a 6 páginas: problema, marco teórico, datos, modelo, resultados, verificación, recomendación y referencias (formato APA).
4. Aplicación web publicada y su enlace de acceso.
5. Cuaderno **analisis.ipynb** ejecutado, con el tratamiento de datos (faltantes, atípicos, agregaciones) documentado.
6. Resumen ejecutivo de una página dirigido a la persona que tomaría la decisión.

Entrega del proyecto terminado en el repositorio del curso

El proyecto terminado se sube al repositorio del curso en GitHub, donde se reúnen los proyectos de todos los estudiantes:

https:

`//github.com/Franciscocamposk/PROYECTOS_CALCULO_DIFERENCIAL_E_INTEGRAL.git`

1. Hacer un *fork* del repositorio y trabajar en él durante las semanas del proyecto.
2. Colocar todo el trabajo en la carpeta `PROYECTO_2/<código>_<apellido>/` (por ejemplo, `PROYECTO_2/C-01_AGUIRRE/`) con la estructura mínima: datos, código, pruebas, cuaderno, ficha técnica (PDF), resumen ejecutivo (PDF), diapositivas y un `README.md` con el enlace a la aplicación web.
3. Enviar un *pull request* al repositorio del curso antes de la fecha de la presentación. Solo se evalúa lo que esté en el repositorio a esa fecha; no se reciben entregas por correo.

Día de la presentación

La presentación y defensa es una exposición ante un cliente: cada estudiante debe asistir con **vestimenta formal** y llegar con la aplicación web y las diapositivas listas para proyectar.

Cronograma y revisiones de avance

Semana	Actividad	Evidencia de avance
10	Recepción del proyecto y de la base; exploración inicial y pregunta de decisión.	Propuesta de una página.
11	Lectura y limpieza en Python (<code>limpieza.py</code>); gráficos exploratorios.	Repositorio creado y diccionario de variables.
12	Modelo, integrales y verificación (<code>modelo.py</code>) con sus pruebas.	Borrador de la ficha técnica y pruebas que pasan.
13	Prototipo de la aplicación web con el escenario base.	Enlace funcional.
14	Escenarios y sensibilidad (<code>escenarios.py</code>); recomendación.	Resumen ejecutivo preliminar.
15	Presentación y defensa individual, con vestimenta formal.	<i>Pull request</i> al repositorio del curso.

Las revisiones de las semanas 10 a 14 son formativas: permiten corregir el rumbo y no suman puntos por sí mismas, pero la propuesta de la semana 10 es requisito para continuar.

PROYECTO 2 | CEREBRO Y FARMACOLOGÍA

Cafeína, atención y sueño en semana de exámenes

Asignado a: VILLALOBOS VILLALTA KENDALL FABRICIO

2

Contenidos del curso: **Modelo exponencial** **Área bajo la curva** **Integral impropia** **Superposición**

Cliente. Coordinación de Vida Estudiantil de la Sede del Sur (UCR, Golfito), que prepara una campaña sobre el consumo de café y bebidas energéticas durante la semana de exámenes.

Problema. La cafeína bloquea la adenosina en el cerebro y mejora la atención, pero la velocidad con que se elimina varía mucho entre personas: fumar la acelera y algunos anticonceptivos la retrasan. Tomarla tarde empeora el sueño, que es clave para la memoria. Vida Estudiantil quiere recomendaciones basadas en datos y diferenciadas según la velocidad de eliminación.

Pregunta de decisión. ¿Cuántas tomas, de qué bebida y hasta qué hora conviene consumir cafeína para tener más horas de atención en la jornada de estudio sin afectar el sueño?

Criterio de éxito. Para cada tipo de persona (eliminación rápida, normal y lenta) se recomienda el plan con más horas de concentración eficaz entre las 8:00 y las 20:00, con concentración al acostarse menor que el umbral y consumo diario dentro del máximo.

Datos entregados (Excel)

- Concentración de cafeína de 60 voluntarios en 13 momentos durante 24 h tras tomar 200 mg (780 registros).
- Características de los voluntarios y 480 pruebas de tiempo de reacción.
- Cafeína de seis bebidas, seis planes de consumo, constante de absorción y umbrales de efecto y de sueño.

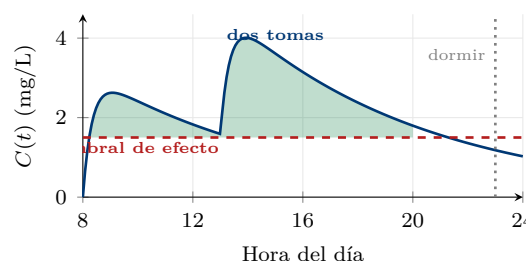


Ilustración cualitativa del modelo.

Modelo.

$$C(t) = \frac{D k_a}{V(k_a - k)}(e^{-kt} - e^{-k_a t}), \quad AUC = \int_0^\infty C(t) dt = \int_0^{24} C dt + \frac{C(24)}{k} = \frac{D}{V k},$$

$$C_{\text{plan}}(t) = \sum_j C(t - t_j; D_j), \quad H_{\text{ef}} = \int_8^{20} \mathbf{1}[C_{\text{plan}}(t) \geq c_{\text{ef}}] dt.$$

Símbolo	Significado	Unidades
$C(t)$	Concentración de cafeína en sangre	mg/L
D	Cafeína de una toma	mg
k_a, k	Constantes de absorción (dada) y de eliminación	1/h
V	Volumen de distribución	L
$t_{1/2} = \ln 2/k$	Vida media	h
H_{ef}	Horas con efecto durante la jornada de estudio	h

Marco teórico mínimo (diapositivas y ficha técnica). Decrecimiento exponencial y vida media; modelo de un compartimento con absorción; integral impropia y convergencia; área bajo la curva y depuración; principio de superposición.

Trabajo requerido.

- Para cada voluntario de **Concentraciones**, estimar k con la pendiente de $\ln C$ en la fase final (desde las 6 h), la vida media y el AUC con la regla del trapecio más la cola impropia $\int_{24}^\infty C dt \approx C(24)/k$.
- Calcular la depuración $CL = D/AUC$ y el volumen $V = CL/k$; medir qué porcentaje del AUC aporta la cola.

3. Comparar la vida media según tabaquismo y anticonceptivos (hoja **Voluntarios**) y clasificar a las personas en eliminación rápida, normal y lenta (terciles).
4. Relacionar el tiempo de reacción con la concentración (hoja **Pruebas_atencion**).
5. Evaluar por superposición los planes de **Planes** para cada grupo: horas con efecto (integral de la función indicadora), concentración a la hora de dormir y cafeína total.

Parte programada en Python

- `analizar_voluntario(t, c, dosis)`: k , vida media, AUC con trapecio más cola, CL y V .
- `clasificar(voluntarios)`: grupos por terciles de vida media.
- `concentracion_plan(plan, parametros, t)`: superposición vectorizada con `numpy`.
- `horas_efecto(t, c, umbral, inicio, fin)`: integral de la función indicadora.
- `evaluar_planes(...)`: tabla de planes por grupo con su cumplimiento.

Validación obligatoria: integrando numéricamente el modelo $C(t)$ de cada voluntario hasta 200 h se recupera $D/(Vk)$ con error menor que 0,5 %, y $\int_0^\infty e^{-kt} dt = 1/k$ se verifica con la integral truncada.

Aplicación web	Simulador de consumo: la persona elige bebidas y horas, indica su tipo de eliminación y ve la curva de cafeína, la franja con efecto y la concentración al acostarse.
Solución	Guía de consumo por tipo de persona (plan, hora límite de la última toma y máximo diario) para la campaña.
Indicadores	Horas con efecto, concentración a las 23:00, cafeína diaria y mejora del tiempo de reacción.

Proyecto académico con datos simulados; sus resultados no constituyen una recomendación médica.

Sobre los datos entregados

Archivo de datos de cada estudiante

Cada estudiante recibe un archivo de Excel con su base personal; cada hoja contiene una tabla de datos y los nombres de las columnas indican la variable y sus unidades. Las bases fueron construidas con patrones y órdenes de magnitud realistas del contexto costarricense; los lugares son reales y los datos, simulados. Como en cualquier registro real, pueden incluir algunos valores faltantes o atípicos que deberán detectarse y tratarse de forma justificada.

Estructura de la presentación

N.º	Contenido	Tiempo aprox.
1	Organización, persona usuaria, problema y recomendación en una frase.	1 min
2	Pregunta de decisión y criterio de éxito.	0,5 min
3	Marco teórico: conceptos de cálculo integral y del área de aplicación, fórmulas usadas y su justificación (deducción breve o teorema).	2 min
4	Datos: exploración, patrones, faltantes, atípicos y tratamiento.	1 min
5	Variables, unidades, supuestos, restricciones y construcción del modelo.	1,5 min
6	Integración, solución y verificación.	1,5 min
7	Demostración de la aplicación web y recorrido breve por el código en Python (pruebas que pasan).	2,5 min
8	Escenario actual, alternativas, sensibilidad y comparación cuantificada.	1,5 min
9	Solución recomendada, implementación, riesgos y seguimiento.	1,5 min
10	Referencias bibliográficas.	—

Rúbrica de evaluación

Criterio	Peso	Excelente (100 %)	Aceptable (60 %)	Insuficiente (0–30 %)
Comprensión y delimitación del problema	10 % 1,10 pts	Contexto y persona usuaria claros; pregunta de decisión precisa.	Problema comprendido pero difuso o poco delimitado.	Problema mal comprendido o sin pregunta de decisión.
Exploración y tratamiento de los datos	12 % 1,32 pts	Patrones identificados; faltantes y atípicos tratados con criterio explícito; unidades claras.	Tratamiento parcial o poco documentado.	Datos usados sin revisar o mal procesados.
Marco teórico y construcción del modelo	18 % 1,98 pts	Marco teórico correcto y pertinente en las diapositivas y la ficha; modelo justificado por la teoría y los datos, con unidades, dominio y restricciones.	Marco teórico incompleto o modelo con justificación parcial.	Sin marco teórico o modelo arbitrario.
Integración, procedimientos y corrección matemática	18 % 1,98 pts	Integrales correctas, método justificado, unidades e interpretación claras.	Errores menores o interpretación incompleta.	Errores conceptuales o integrales sin interpretar.
Verificación, validación y sensibilidad	10 % 1,10 pts	Contraste con los acumulados de la base y tres escenarios analizados.	Verificación parcial o menos escenarios.	Sin verificación ni escenarios.
Programación en Python y aplicación web	15 % 1,65 pts	Funciones pedidas implementadas, documentadas y con pruebas que pasan en la integración continua; cuaderno reproducible; aplicación publicada, interactiva y centrada en la decisión.	Código funcional pero incompleto, sin pruebas o poco documentado; aplicación con limitaciones.	El código no se ejecuta, no es en Python o no es propio; aplicación inexistente o no interactiva.
Viabilidad, impacto y claridad de la solución	12 % 1,32 pts	Recomendación cuantificada, aplicable y con condiciones explícitas.	Recomendación general o poco cuantificada.	Sin recomendación o sin relación con el análisis.
Presentación y defensa individual	5 % 0,55 pts	Diapositivas claras y profesionales, vestimenta formal, dominio total del modelo y respuestas precisas.	Dominio parcial.	No explica su propio trabajo.
Total	100 % 11 pts			

Debajo de cada peso: puntos sobre el 11 % de la nota final. Nota del proyecto = porcentaje obtenido $\times$ 0,11.

Criterio de una buena solución

Una buena solución deberá indicar **qué** debe hacerse, **quién** podría hacerlo, **cuándo**, **con qué recursos**, **cuánto** costaría o ahorraría, qué resultado se espera, bajo qué supuestos funciona y qué indicador permitiría verificar su efecto. Recomendaciones generales como “mejorar el servicio”, “reducir costos” o “aumentar ventas” no serán suficientes.

Preguntas de la defensa individual

1. ¿Qué patrones encontró en los datos y cómo trató faltantes y atípicos?
2. ¿Qué resultado teórico (teorema, propiedad de la integral o ley física) sustenta su modelo?
3. ¿Por qué el modelo representa razonablemente el proceso?
4. ¿Qué significa la integral y cuáles son sus unidades?
5. ¿Cómo se verificó el resultado?
6. ¿Qué hace cada función principal de su código y cómo comprobó con pruebas que es correcta?
7. ¿Qué parámetro modifica más la decisión?
8. ¿Qué solución concreta se propone y cuál sería su impacto?
9. ¿Qué tendría que ocurrir para que la recomendación deje de ser válida?
10. ¿Cómo se mediría el resultado después de implementar la solución?

Preguntas específicas de su proyecto

1. ¿Por qué el área bajo la curva requiere una integral impropia y cuánto aporta la cola C_n/k ?
2. ¿Por qué $AUC = D/(Vk)$ y qué se valida con esa fórmula?
3. ¿Por qué la recomendación cambia entre personas de eliminación rápida y lenta?

Declaración de autoría (adjuntar a la ficha técnica)

Declaro que este proyecto fue elaborado de forma individual con la base de datos que me fue asignada, que todo tratamiento de los datos está documentado y que el código y las visualizaciones son de mi autoría o están debidamente citados.

Nombre completo

Carné

Firma